

Mini Internet Protocol Stack Simulator Report

Group Members

chunyu zheng 24279373

Fubin Qiu 23673779

1. Overview

The simulator shows how Layer 2, Layer 3, and Layer 4 work together to send data from Host A to Host B through Router R1.

2. Network Topology and Configuration

The simulator uses the fixed topology defined in the project specification. Host A and Host B are connected through Router R1. The network contains two subnets.

Network 1: 10.0.1.0/24

Network 2: 10.0.2.0/24

IP addresses:

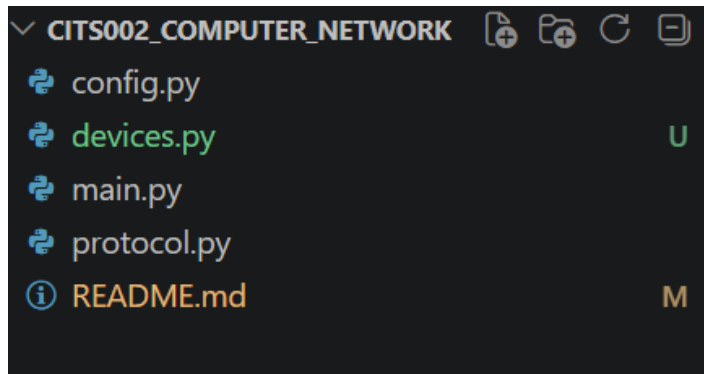
- Host A: 10.0.1.10
- Router R1 Interface 1: 10.0.1.1
- Router R1 Interface 2: 10.0.2.1
- Host B: 10.0.2.20

MAC addresses:

- Host A: AA:AA:AA:AA:AA:AA
- Router R1 Interface 1: BB:BB:BB:BB:BB:BB
- Router R1 Interface 2: CC:CC:CC:CC:CC:CC
- Host B: DD:DD:DD:DD:DD:DD

2. File Structure

The project contains five main files:



- main.py is the entry file of the entire system. It is responsible for parsing the command-line parameters to obtain the size of the application layer message (message size), initializing the network topology structure (Host A, Host B, Router R1), and triggering the data transmission process based on the deterministic environment.
- config.py stores fixed values used by the simulator.
- devices.py contains the device classes.
- protocol.py contains the protocol header classes.

3. Protocol Design

3.1 Layer 2: frame

- Layer 2 uses an Ethernet-like frame.
- The Frame class stores the Layer 2 header fields, including destination MAC address, source MAC address, type, and payload.
- The payload of the frame is a layer 3 packet
- Layer 2 is responsible for frame creation, MAC address lookup, MAC learning, frame forwarding, and delivering the packet to Layer 3.
- The receiver checks the destination MAC address and Ethernet type before passing the packet to Layer 3. Invalid frames are dropped.

3.2 Layer 3: packet

- Layer 3 uses an IP-like packet.
- The Packet class stores the source IP, destination IP, TTL, protocol, total length, and payload.
- The payload is a Layer 4 segment. When the packet passes through Router R1, the TTL is decreased by 1. If the TTL reaches 0, the packet is dropped and will not be forwarded.
- Layer 3 is used for IP addressing, routing, forwarding, and delivering the segment to Layer 4 at the destination host.
- Router R1 performs routing table lookup and selects the outgoing interface before forwarding the packet.

3.3 Layer 4:segment

If the message is larger than 500 bytes, it is split into multiple segments. Each segment is sent sequentially using the rdt2.2 alternating bit protocol. The sender waits for the correct ACK before sending the next segment.

The header of Layer 4 includes the source port, destination port, length, checksum, type(0 represents DATA, 1 represents ACK), sequence number, data payload. To provide error detection functionality, the system packages and calculates the checksum for each segment before sending it. The receiving end verifies the checksum to ensure the integrity of the data and discards all damaged segments.

4. Device Design

4.1 Host

The Host class is used to represent both Host A and Host B.

Each host stores its own IP address, MAC address, routing table, and MAC lookup table. These values are used when the host needs to send data to another device.

When a host sends a segment, it first creates a Layer 3 packet. Then it checks the routing table to find the next-hop IP address. After that, it uses the MAC lookup table to find the correct destination MAC address and creates a Layer 2 frame.

When a host receives a frame, it learns the source MAC address and takes out the packet from the frame. If the destination IP address matches the host's own IP address, the host delivers the payload to Layer 4.

In this project, Host A mainly sends the data, while Host B receives the data and sends ACK segments back.

In the Layer 4 operation, Host acts as the transport layer endpoint. Since the maximum data capacity of a UDP-like segment is limited to 500 bytes, when the message size generated by the application layer exceeds this limit, the Host will split the data into multiple fragments (chunks). The Host uses the rdt2.2 protocol (alternating bit protocol) to ensure the reliable delivery of data. Specifically, the sender assigns a sequence number(0 or 1) to each DATA segment and alternates strictly. After sending the data, the sender will wait for the correct ACK in a loop. If it receives an incorrect or duplicate ACK, the sender will retransmit the current segment. On the receiving end, once a fully correct DATA segment with the expected sequence number is received, the data will be passed up to the application layer and an ACK with the corresponding sequence number will be returned. If the receiving end finds that the segment checksum verification fails or the sequence number is duplicated, it will resend the previous ACK.

4.2 Router

The Router class is used to represent Router R1.

Router R1 has two interfaces. Interface 1 is connected to Network 1, and Interface 2 is connected to Network 2. Each interface has its own IP address and MAC address.

When the router receives a frame, it learns the source MAC address and extracts the packet. It then reads the destination IP address, decreases the TTL, and checks the routing table to decide where the packet should go next.

After the routing decision is made, the router selects the outgoing interface and finds the next-hop MAC address. It then creates a new Layer 2 frame and forwards it to the correct host.

The router only handles Layer 2 and Layer 3 forwarding. It does not process the Layer 4 data, checksum, or ACK logic.

5. Data Transmission Flow

Host A creates a Layer 4 DATA segment. It is then wrapped into a Layer 3 packet and a Layer 2 frame.

Router R1 receives the frame, checks the destination IP, decreases the TTL, and forwards it to Host B

- application data → layer4 segment → layer3 packet → layer2 frame → router r1 → B

After B receives a valid data segment, it sends an ACK segment to A.

- B → router r1 → A

If the message is larger than 500 bytes, it is split into multiple segments. Host A sends the next segment only after receiving the correct ACK.

During this transmission process, if the receiving end (Host B) detects a checksum error or finds duplicate sequence numbers, it will directly discard the segment and resend the last valid ACK. The sending end (Host A) will remain in a waiting state until it receives the expected ACK. If necessary, it will trigger the retransmission mechanism. Only when all data fragments have been successfully sent via the rdt2.2 protocol and the ACKs are received in sequence, can the entire data delivery process be considered complete.

6. Testing

The program was tested with 10 500 501 1200 message sizes

For 10, the output shows one DATA segment with sequence number 0 sent from Host A to Host B. Host B sends back one ACK with sequence number 0. This shows that basic end-to-end delivery works.

For 500, the message is sent as one segment because 500 bytes is the maximum data size for one UDP-like segment.

For 501, the message is split into two segments. The sequence numbers are 0 and 1. This shows that segmentation works correctly.

For 1200, the message is split into three segments. The sequence numbers are 0, 1, and 0. This shows that the alternating-bit protocol works correctly.

Overall, the tests show that the simulator can deliver data from A to B, return ACKs from B to A, and handle messages larger than 500 bytes.

Command	Expected Result	Result
python main.py 10	DATA seq=0 and ACK seq=0	Passed
python main.py 500	One DATA segment only	Passed
python main.py 501	Two segments, seq=0 and seq=1	Passed
python main.py 1200	Three segments, seq=0, seq=1, seq=0	Passed

```

Host B: Layer 4: Checksum verified
Host B: Layer 4: DATA segment delivered to Application Layer. Data size =10
Host B: Layer 4: Segment created by adding transport layer header (ACK, seq=0)
Host B: Layer 4: Segment sent to Network Layer
Host B: Layer 3: Segment received from Transport Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=100
Host B: Layer 3: Destination IP read: 10.0.1.10
Host B: Layer 3: Routing table lookup performed
Host B: Layer 3: Next-hop IP determined: 10.0.2.1
Host B: Layer 3: Outgoing interface selected
Host B: Layer 3: Packet forwarded to Data Link Layer
Host B: Layer 2: Packet received from Network Layer
Host B: Layer 2: Destination MAC lookup for next-hop IP (10.0.2.1) → CC:CC:CC:CC:CC:CC
Host B: Layer 2: Frame created: SRC_MAC=DD:DD:DD:DD:DD:DD, DST_MAC=CC:CC:CC:CC:CC:CC
Host B: Layer 2: Frame sent
Router R1: Layer 2: Frame received on Interface 2
Router R1: Layer 2: Source MAC learned: DD:DD:DD:DD:DD:DD on Interface 2
Router R1: Layer 2: Packet delivered to Network Layer
Router R1: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=100
Router R1: Layer 3: Destination IP read: 10.0.1.10
Router R1: Layer 3: TTL decremented: 100 → 99
Router R1: Layer 3: Routing table lookup performed
Router R1: Layer 3: Next-hop IP determined: 10.0.1.10
Router R1: Layer 3: Outgoing interface selected (Interface 1)
Router R1: Layer 3: Packet forwarded to Data Link Layer
Router R1: Layer 2: Packet received from Network Layer
Router R1: Layer 2: Destination MAC lookup for next-hop IP (10.0.1.10) → AA:AA:AA:AA:AA:AA
Router R1: Layer 2: Frame created: SRC_MAC=BB:BB:BB:BB:BB:BB, DST_MAC=AA:AA:AA:AA:AA:AA
Router R1: Layer 2: Frame forwarded on Interface 1
Host A: Layer 2: Frame received
Host A: Layer 2: Source MAC learned: BB:BB:BB:BB:BB:BB
Host A: Layer 2: Packet delivered to Network Layer
Host A: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=99
Host A: Layer 3: Destination IP read: 10.0.1.10
Host A: Layer 3: Packet identified as local delivery
Host A: Layer 3: Segment delivered to Transport Layer
Host A: Layer 4: Segment received from Network Layer
Host A: Layer 4: Checksum verified
Host A: Layer 4: ACK received: seq=0

```

Figure 1. Output for the 10-byte test. Host A receives ACK seq=0, which shows that the DATA segment was delivered successfully.

```

Host B: Layer 4: DATA segment delivered to Application Layer. Data size =500
Host B: Layer 4: Segment created by adding transport layer header (ACK, seq=0)
Host B: Layer 4: Segment sent to Network Layer
Host B: Layer 3: Segment received from Transport Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=100
Host B: Layer 3: Destination IP read: 10.0.1.10
Host B: Layer 3: Routing table lookup performed
Host B: Layer 3: Next-hop IP determined: 10.0.2.1
Host B: Layer 3: Outgoing interface selected
Host B: Layer 3: Packet forwarded to Data Link Layer
Host B: Layer 2: Packet received from Network Layer
Host B: Layer 2: Destination MAC lookup for next-hop IP (10.0.2.1) → CC:CC:CC:CC:CC:CC
Host B: Layer 2: Frame created: SRC_MAC=DD:DD:DD:DD:DD:DD, DST_MAC=CC:CC:CC:CC:CC:CC
Host B: Layer 2: Frame sent
Router R1: Layer 2: Frame received on Interface 2
Router R1: Layer 2: Source MAC learned: DD:DD:DD:DD:DD:DD on Interface 2
Router R1: Layer 2: Packet delivered to Network Layer
Router R1: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=100
Router R1: Layer 3: Destination IP read: 10.0.1.10
Router R1: Layer 3: TTL decremented: 100 → 99
Router R1: Layer 3: Routing table lookup performed
Router R1: Layer 3: Next-hop IP determined: 10.0.1.10
Router R1: Layer 3: Outgoing interface selected (Interface 1)
Router R1: Layer 3: Packet forwarded to Data Link Layer
Router R1: Layer 2: Packet received from Network Layer
Router R1: Layer 2: Destination MAC lookup for next-hop IP (10.0.1.10) → AA:AA:AA:AA:AA:AA
Router R1: Layer 2: Frame created: SRC_MAC=BB:BB:BB:BB:BB:BB, DST_MAC=AA:AA:AA:AA:AA:AA
Router R1: Layer 2: Frame forwarded on Interface 1
Host A: Layer 2: Frame received
Host A: Layer 2: Source MAC learned: BB:BB:BB:BB:BB:BB
Host A: Layer 2: Packet delivered to Network Layer
Host A: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=99
Host A: Layer 3: Destination IP read: 10.0.1.10
Host A: Layer 3: Packet identified as local delivery
Host A: Layer 3: Segment delivered to Transport Layer
Host A: Layer 4: Segment received from Network Layer
Host A: Layer 4: Checksum verified
Host A: Layer 4: ACK received: seq=0
Host A: Layer 4: Checksum computed
Host A: Layer 4: Segment created by adding transport layer header (DATA, seq=1) (encapsulation)
Host A: Layer 4: Segment sent to Network Layer

```

Figure 2.1. First segment of the 501-byte test. The first 500-byte DATA segment uses sequence number 0, and Host A receives ACK seq=0 before sending the next segment.

```

Host B: Layer 4: DATA segment delivered to Application Layer. Data size =1
Host B: Layer 4: Segment created by adding transport layer header (ACK, seq=1)
Host B: Layer 4: Segment sent to Network Layer
Host B: Layer 3: Segment received from Transport Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=100
Host B: Layer 3: Destination IP read: 10.0.1.10
Host B: Layer 3: Routing table lookup performed
Host B: Layer 3: Next-hop IP determined: 10.0.2.1
Host B: Layer 3: Outgoing interface selected
Host B: Layer 3: Packet forwarded to Data Link Layer
Host B: Layer 2: Packet received from Network Layer
Host B: Layer 2: Destination MAC lookup for next-hop IP (10.0.2.1) → CC:CC:CC:CC:CC:CC
Host B: Layer 2: Frame created: SRC_MAC=DD:DD:DD:DD:DD:DD, DST_MAC=CC:CC:CC:CC:CC:CC
Host B: Layer 2: Frame sent
Router R1: Layer 2: Frame received on Interface 2
Router R1: Layer 2: Source MAC learned: DD:DD:DD:DD:DD:DD on Interface 2
Router R1: Layer 2: Packet delivered to Network Layer
Router R1: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=100
Router R1: Layer 3: Destination IP read: 10.0.1.10
Router R1: Layer 3: TTL decremented: 100 → 99
Router R1: Layer 3: Routing table lookup performed
Router R1: Layer 3: Next-hop IP determined: 10.0.1.10
Router R1: Layer 3: Outgoing interface selected (Interface 1)
Router R1: Layer 3: Packet forwarded to Data Link Layer
Router R1: Layer 2: Packet received from Network Layer
Router R1: Layer 2: Destination MAC lookup for next-hop IP (10.0.1.10) → AA:AA:AA:AA:AA:AA
Router R1: Layer 2: Frame created: SRC_MAC=BB:BB:BB:BB:BB:BB, DST_MAC=AA:AA:AA:AA:AA:AA
Router R1: Layer 2: Frame forwarded on Interface 1
Host A: Layer 2: Frame received
Host A: Layer 2: Source MAC learned: BB:BB:BB:BB:BB:BB
Host A: Layer 2: Packet delivered to Network Layer
Host A: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=99
Host A: Layer 3: Destination IP read: 10.0.1.10
Host A: Layer 3: Packet identified as local delivery
Host A: Layer 3: Segment delivered to Transport Layer
Host A: Layer 4: Segment received from Network Layer
Host A: Layer 4: Checksum verified
Host A: Layer 4: ACK received: seq=1

```

Figure 2.2. Second segment of the 501-byte test. The remaining 1-byte DATA segment uses sequence number 1, and Host A receives ACK seq=1.

Overall, the tests show that the simulator works for different message sizes. The data can be sent from Host A to Host B, and the ACK can be sent back from Host B to Host A. For messages larger than 500 bytes, the program can split the data into multiple segments and use sequence numbers 0 and 1 correctly.

7. Output and Logging

The program prints structured logs for each layer. The logs show the full process of sending data from A to B and returning an ACK from B to A.

Layer 2 logs show frame creation, frame reception, MAC address lookup, MAC learning, frame forwarding, and packet delivery to the Network Layer.

Layer 3 logs show packet reception, destination IP reading, routing table lookup, next-hop IP selection, outgoing interface selection, TTL decrement, and local delivery.

Layer 4 logs show data reception from the application layer, checksum computation, checksum verification, DATA segment delivery, ACK creation, and ACK reception.

For example, when running `python main.py 10`, the output shows that A sends one DATA segment with sequence number 0 to B. B verifies the checksum, delivers the data to the application layer, and sends an ACK with sequence number 0 back to A. This confirms that the logging output matches the expected Layer 2, Layer 3, and Layer 4 behaviour.